

Policy Search Engine

Week 1 Book - From Zero Understanding to Project Demo

Prepared for: Sowmya Lakshmi.B

What this book is

This is a beginner-friendly book for explaining your project in a simple, human way. It teaches the idea, the words, the code, the viva answers, the demo flow, and the week 2 preview.

How to use it

Read the story box first, then the technical box, then the say-it-to-ma'am box. That way your brain remembers both understanding and interview wording.

A small coder promise

Build to solve real problems. Learn calmly. Ask why before what. Keep health and discipline. Money should be the result of value, not the only target.

Index

- 1. The project mission - what problem it solves
- 2. The big words beginners ask about: technology, framework, library, model, API, database
- 3. Python journey through this project
- 4. From PDF to answer: parsing, chunking, embeddings, semantic search
- 5. Hallucination filtering, threshold, and human-like response
- 6. Code map: imports, functions, and the chat UI
- 7. What Week 1 means from the official sheet
- 8. How to explain the demo to ma'am
- 9. Viva questions and safe answers
- 10. Common mistakes and honest corrections
- 11. Week 2 preview from the official source

Important truth

This book stays honest. Where something is not yet implemented in your current code, it is marked as future work instead of pretending it is already done.

Chapter 1 - What is this project really?

The project is a semantic AI-based policy search engine. It helps a user search company policy PDFs in a smarter way than manual reading.

Simple story

Imagine a library with many policy pages. Instead of opening every page by hand, you ask one question and the system finds the closest policy meaning for you.

Technical meaning

This is called closed-domain semantic retrieval because the chatbot answers only from the uploaded documents, not from the whole internet.

What to say to ma'am

Ma'am, the system searches only inside the uploaded policy document and gives meaning-based answers from that document.

Chapter 2 - The beginner words that look scary

Many tech words sound difficult only because nobody explains the purpose first. Here is the easiest way to understand them.

Word	Easy meaning	Why it exists
Programming language	A language used to write instructions for computers.	So humans can tell computers what to do.
Library	A ready-made set of useful code written by others.	So we do not build everything from zero.
Framework	A bigger helper structure that gives rules and ready-made patterns.	So apps can be built faster and in an organized way.
Technology	A tool or method used to solve a problem.	So real-world work becomes possible.
Model	A trained AI brain that learned patterns from data.	So the computer can understand or generate something useful.
API	A messenger that carries requests and responses.	So different parts of software can talk to each other.
Database	A place where data is stored and managed.	So information can be saved and found later.

Streamlit in one line

Streamlit is a Python framework for quickly making web interfaces. In this project we use it to build the chatbot page.

FastAPI in one line

FastAPI is a backend framework for building APIs. In the current demo it is a future recommended tool, not the main one you used.

LangChain in one line

LangChain is an orchestration framework for connecting prompts, models, tools, and retrieval. It is recommended in the project sheet, but your current demo focuses on the core retrieval pipeline.

FAISS / ChromaDB in one line

FAISS and ChromaDB are vector databases / vector search tools for storing embeddings efficiently. They are a future improvement for your current app.

Chapter 3 - How Python became your first tool

You learned Python from scratch by connecting every small idea to the same project. That is why the learning feels connected instead of random.

Python step	What it means	Why it mattered in this project
print()	Show output on screen.	Used to test first programs and show answers.
input()	Take text from the user.	Used for asking questions in chatbot flow.
if / else	Make a decision.	Used for pass/fail, exit, or answer/no-answer logic.
loops	Repeat work many times.	Used to check many lines, many chunks, many results.
lists	Store many items together.	Used for policies, chunks, results, and messages.
functions	Small reusable pieces of code.	Used to read PDF, chunk text, and search answers.
file handling	Open and read external files.	Used to read policy.txt and policy.pdf.

Why you used 'python test.py' before

Because Python is the interpreter that runs your .py file. The file itself is only code text; Python is the tool that executes it.

Why Streamlit uses 'streamlit run app.py'

Streamlit needs its own runner because it opens an interactive web app, not a normal command-line program.

Memory note

Code is easier to remember when each new idea solves one real problem in the project.

Chapter 4 - From PDF to answer

This is the heart of the project. A PDF is turned into searchable knowledge, then the chatbot finds the closest answer meaning.

Step 1 - PDF reading

The system reads the policy PDF and extracts its text using pypdf.

Step 2 - Parsing

Parsing means pulling usable text out of the PDF pages so the computer can work with it.

Step 3 - Cleaning

Extra spaces and messy line breaks are removed so the text becomes easier to process.

Step 4 - Chunking

The long document is split into smaller pieces called chunks. This helps the search become more accurate.

Step 5 - Embeddings

Each chunk is converted into a numerical vector. This is how the computer stores the meaning of text in math form.

Step 6 - Semantic search

The question is also converted into a vector, then the system compares meaning similarity instead of only exact words.

Step 7 - Threshold filtering

If the answer confidence is too low, the system rejects it instead of giving a random response.

Step 8 - Human-like response

The final answer is shown in a friendly sentence so the chatbot feels conversational.

Knowledge base

The knowledge base is the internal searchable semantic data built from the uploaded PDF chunks and embeddings.

Important honesty

In your current code, the knowledge base is stored temporarily in memory during runtime, not in a permanent vector database yet.

Chapter 5 - The code map, explained simply

If ma'am asks about the code, explain it in blocks. You do not need to explain every symbol separately.

Imports

Import	What to say
re	Used for cleaning text and splitting sentences with patterns.
hashlib	Used to create a unique fingerprint of the uploaded PDF so the app can detect a new file.
BytesIO	Used to read the uploaded PDF bytes in memory without saving it first.
streamlit as st	Used for the chatbot webpage UI.
PdfReader	Used to extract text from PDF pages.
SentenceTransformer	Used to create embeddings from text.
cosine_similarity	Used to compare embeddings mathematically.

Key functions

Function	Meaning
clean_text()	Removes extra spaces and makes text neat.
split_into_chunks()	Breaks big PDF text into smaller pieces.
load_model()	Loads the pretrained Sentence Transformer model.
extract_text_from_pdf()	Reads text from PDF pages.
build_knowledge_base()	Creates embeddings for all chunks.
smart_reply()	Handles simple greetings like hi, thanks, bye.
search_answer()	Finds the best matching chunk using semantic similarity and threshold checking.

```
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
```

How to explain the main search function

User question -> question embedding -> cosine similarity -> best matching chunk -> threshold filter -> human-like answer.

What is normalize_embeddings?

It makes embeddings easier to compare fairly when cosine similarity is used.

What is the current limitation?

The current app is a semantic retrieval chatbot. It is not full LLM generation yet.

Chapter 6 - What Week 1 means in the official sheet

This chapter connects directly to the project sheet so you can explain why Week 1 counts as real implementation.

Week 1 item	Simple meaning	What you can show
Create the knowledge base	Turn the uploaded PDF into searchable chunks and embeddings.	Show the PDF upload and the AI reading process.
Upload / parse documents	Read the PDF and extract its text.	Show that the chatbot can read the file.
Build a basic chat interface	Create a page where a user can ask questions.	Open the Streamlit chatbot UI in the browser.
Test document loading	Check whether PDF content is successfully extracted.	Upload the file and confirm the app starts.
Test chunking	Confirm the document is split into smaller pieces.	Tell her that chunks are created internally for search accuracy.
Test baseline Q&A; flow	Ask sample questions and get related answers.	Use remote work, leave policy, medical insurance.

What Week 1 does NOT need

It does not need perfect enterprise-grade AI, long-term memory, full vector databases, or advanced LLM generation.

How to describe Week 1 politely

Ma'am, I completed the first working retrieval version: PDF reading, chunking, embeddings, semantic search, threshold filtering, and chat UI.

Marks reminder

Week 1 implementation: 10 marks. Week 1 testing: 5 marks.

Chapter 7 - How to demo it in front of ma'am

A good demo is not about speaking fast. It is about showing the right things in the right order.

Suggested demo flow

- Open the project in VS Code.
- Show the app.py file and the PDF file.
- Run the Streamlit app.
- Upload the policy PDF.
- Ask 3 to 5 safe semantic questions.
- Show one low-confidence query to prove filtering.
- Explain the flow slowly.

Safe demo questions

remote work, work from home, leave policy, vacation, medical insurance, travel reimbursement, office timing

Questions to avoid during demo

Very random queries like banana spaceship, or edge cases that may retrieve an imperfect chunk.

What to say if a result is not perfect

Semantic retrieval sometimes ranks a nearby policy chunk higher than the ideal one. That is a normal improvement area, not a failure.

Short viva answers

Question	Short answer
What is embedding?	Text meaning converted into numerical vectors.
What is semantic search?	Search by meaning instead of exact keywords.
What is threshold?	Minimum confidence needed to accept an answer.
What is hallucination filtering?	Rejecting weak or random answers.
Why Streamlit?	Fast UI for AI apps using Python.
Why scikit-learn?	To compute cosine similarity.

Chapter 8 - Honest limitations and how to answer them

Honesty helps more than fake perfection. If the evaluator asks about limitations, say them clearly.

Current limitation

The current app stores embeddings temporarily in memory. It does not yet use a persistent vector database.

Why answer may sometimes look wrong

Embedding similarity is good but not perfect. A nearby chunk can sometimes rank above the exact expected chunk.

How to answer that in viva

This is a retrieval-based semantic chatbot. Future work can improve ranking using ChromaDB, FAISS, and LLM-based reranking.

What not to say

Do not claim full RAG or full production-grade database support if it is not yet implemented.

Best honest line

Current implementation successfully demonstrates the semantic retrieval pipeline and the basic chatbot workflow.

Chapter 9 - Week 2 preview (truthful, from your source)

This preview is based only on the official project details you shared. It is not invented.

Week 2 item	What it means in simple English
Add retrieval	Improve the system so it finds the most relevant parts more reliably.
Prompt tuning	Adjust the instructions so answers become better and more useful.
Citations	Show where the answer came from in the document.
Admin refresh workflow	A way to reload or update the document knowledge when needed.
Check answer relevance	Test whether the answer really matches the question.
Check citation correctness	Make sure the source shown is the correct one.
Edge queries	Test unusual or tricky questions.
Fallback handling	What the system should say when it does not know the answer.

Week 2 implementation marks

Week 2 implementation: 10 marks. Week 2 testing: 5 marks.

Week 3 preview from the sheet

Improve chat UX, history and logging, access control, final documentation, end-to-end testing, weak-answer fixing, and demo rehearsal.

Chapter 10 - Fast revision cards

Knowledge base

AI-ready searchable data created from the PDF chunks and embeddings.

Document loading

Importing the PDF into the system for processing.

Parsing

Reading the document text from the file.

Chunking

Splitting the document into smaller pieces.

Embedding

Converting text meaning into numerical vectors.

Semantic search

Searching by meaning, not only by exact words.

Cosine similarity

Measuring how close two embeddings are.

Hallucination filtering

Rejecting weak answers to avoid random output.

Threshold

Minimum confidence needed to accept an answer.

Streamlit UI

The browser page where the user chats with the bot.

Session state

Temporary memory used while the app runs.

If you only remember one flow

PDF -> text extraction -> chunking -> embeddings -> similarity search -> threshold check -> answer.

End note

Part 2 of this book can focus on Week 2 implementation in more depth, including retrieval, prompt tuning, citations, refresh workflow, and test cases.

This Week 1 book is meant to help you explain the project clearly, calmly, and honestly during evaluation.